

tf.compat.v1.distribute.MirroredStrategy

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distribute/MirroredStrategy

Synchronous training across multiple replicas on one machine.

Function Signatures

```
tf.compat.v1.distribute.MirroredStrategy( devices=None,  
cross_device_ops=None )
```

Code Examples

```
tf.compat.v1.distribute.MirroredStrategy(  
    devices=None, cross_device_ops=None  
)
```

```
tf.compat.v1.distribute.MirroredStrategy(  
    devices=None, cross_device_ops=None  
)
```

```
strategy = tf.distribute.MirroredStrategy(["GPU:0", "GPU:1"])  
with strategy.scope():  
    x = tf.Variable(1.)  
    x  
    MirroredVariable:{  
    0: ,  
    1:  
    }
```

```
strategy = tf.distribute.MirroredStrategy(["GPU:0", "GPU:1"])
```

```
with strategy.scope():
```

```
x = tf.Variable(1.)
```

```
MirroredVariable:{
```

Documentation

Synchronous training across multiple replicas on one machine.

Inherits From: Strategy

Used in the notebooks

- Migrate single-worker multiple-GPU training

This strategy is typically used for training on one machine with multiple GPUs. For TPUs, use `tf.distribute.TPUStrategy`. To use `MirroredStrategy` with multiple workers, please refer to `tf.distribute.experimental.MultiWorkerMirroredStrategy`.

For example, a variable created under a `MirroredStrategy` is a `MirroredVariable`. If no devices are specified in the constructor argument of the strategy then it will use all the available GPUs. If no GPUs are found, it will use the available CPUs. Note that TensorFlow treats all CPUs on a machine as a single device, and uses threads internally for parallelism.

While using distribution strategies, all the variable creation should be done within the strategy's scope. This will replicate the variables across all the replicas and keep them in sync using an all-reduce algorithm.

Variables created inside a `MirroredStrategy` which is wrapped with a `tf.function` are still `MirroredVariables`.

`experimental_distribute_dataset` can be used to distribute the dataset across the replicas when writing your own training loop. If you are using `.fit` and `.compile` methods available in `tf.keras`, then `tf.keras` will handle the distribution for you.

For example:

Attributes

In general, when using a multi-worker `tf.distribute` strategy such as `tf.distribute.experimental.MultiWorkerMirroredStrategy` or `tf.distribute.TPUStrategy()`, there is a `tf.distribute.cluster_resolver.ClusterResolver` associated with the strategy used, and such an instance is returned by this property.

Strategies that intend to have an associated `tf.distribute.cluster_resolver.ClusterResolver` must set the relevant attribute, or override this property; otherwise, `None` is returned by default. Those strategies should also provide information regarding what is returned by this property.

Single-worker strategies usually do not have a `tf.distribute.cluster_resolver.ClusterResolver`, and in those cases this property will return `None`.

The `tf.distribute.cluster_resolver.ClusterResolver` may be useful when the user needs to access information such as the cluster spec, task type or task id. For example,

For more information, please see `tf.distribute.cluster_resolver.ClusterResolver`'s API docstring.

Methods

`distribute_datasets_from_function`

[View source](#)

Distributes `tf.data.Dataset` instances created by calls to `dataset_fn`.

The argument `dataset_fn` that users pass in is an input function that has a `tf.distribute.InputContext` argument and returns a `tf.data.Dataset` instance. It is expected that the returned dataset from `dataset_fn` is already batched by per-replica batch size (i.e. global batch size divided by the number of replicas in sync) and sharded.

`tf.distribute.Strategy.distribute_datasets_from_function` does not batch or shard the `tf.data.Dataset` instance returned from the input function. `dataset_fn` will be called on the CPU device of each of the workers and each generates a dataset where every replica on that worker will dequeue one batch of inputs (i.e. if a worker has two replicas, two batches will be dequeued from the Dataset every step).

This method can be used for several purposes. First, it allows you to specify your own batching and sharding logic. (In contrast, `tf.distribute.experimental_distribute_dataset` does batching and sharding for you.) For example, where `experimental_distribute_dataset` is unable to shard the input files, this method might be used to manually shard the dataset (avoiding the slow fallback behavior in `experimental_distribute_dataset`). In cases where the dataset is infinite, this sharding can be done by creating dataset replicas that differ only in their random seed.

The `dataset_fn` should take an `tf.distribute.InputContext` instance where information about batching and input replication can be accessed.

You can use `element_spec` property of the `tf.distribute.DistributedDataset` returned by this API to query the `tf.TypeSpec` of the elements returned by the iterator. This can be used to

set the `input_signature` property of a `tf.function`. Follow `tf.distribute.DistributedDataset.element_spec` to see an example.

For a tutorial on more usage and properties of this method, refer to the tutorial on distributed input).

If you are interested in last partial batch handling, read this section.

experimental_distribute_dataset

[View source](#)

Creates `tf.distribute.DistributedDataset` from `tf.data.Dataset`.

The returned `tf.distribute.DistributedDataset` can be iterated over similar to regular datasets.

NOTE: The user cannot add any more transformations to a `tf.distribute.DistributedDataset`. You can only create an iterator or examine the `tf.TypeSpec` of the data generated by it. See API docs of `tf.distribute.DistributedDataset` to learn more.

The following is an example:

Three key actions happening under the hood of this method are batching, sharding, and prefetching.

In the code snippet above, `dataset` is batched by `global_batch_size`, and calling `experimental_distribute_dataset` on it rebatches `dataset` to a new batch size that is equal to the global batch size divided by the number of replicas in sync. We iterate through it using a Pythonic for loop. `x` is a `tf.distribute.DistributedValues` containing data for all replicas,

and each replica gets data of the new batch size.

`tf.distribute.Strategy.run` will take care of feeding the right per-replica data in `x` to the right `replica_fn` executed on each replica.

Sharding contains autosharding across multiple workers and within every worker. First, in multi-worker distributed training (i.e. when you use `tf.distribute.experimental.MultiWorkerMirroredStrategy` or `tf.distribute.TPUStrategy`), autosharding a dataset over a set of workers means that each worker is assigned a subset of the entire dataset (if the right `tf.data.experimental.AutoShardPolicy` is set). This is to ensure that at each step, a